Student Name: HARINI.S

Register Number: 192412098

CO4- ASSIGNMENT 4

TASK 1 – DATA DESIGN REVIEW

Diagnose and Repair a Poor Data Design

1. Aim

To identify the problems in a patient registration system that uses parallel arrays and redesign it using structures and nested structures in C.

2. Explanation

The original program stores patient information in separate arrays such as `name`, `age`, `phone`, `bp`, and `visitDate`. This is called a parallel-array design.

It creates several problems:

1. Related patient details are stored separately, making the program difficult to maintain.
2. `char phone[10]` cannot store a 10-digit phone number and the null character together.
3. Blood pressure contains two values, systolic and diastolic, but the program stores it in one float.
4. The date is stored as text instead of separate day, month, and year values.
5. There is no check to prevent the patient count from exceeding 100.
6. Global variables make the program harder to manage and reuse.
7. Sorting patient details requires moving values in several arrays together.

Improved design: Use a `Patient` structure containing the patient's name, age, phone number, blood pressure, and visit date. Use nested structures for blood pressure and date.

3. Algorithm

1. Start the program.

2. Define a `Date` structure.
3. Define a `BloodPressure` structure.
4. Define a `Patient` structure.
5. Read patient details.
6. Pass the patient information using a structure pointer.
7. Check whether the patient array has available space.
8. Store the patient details.
9. Display the patient information.
10. Stop.

4. C Program

None

```
#include <stdio.h>
#include <string.h>

#define MAX 100

struct Date {
    int day, month, year;
};

struct BloodPressure {
    int systolic, diastolic;
};

struct Patient {
    char name[50];
    int age;
    char phone[11];
    struct BloodPressure bp;
    struct Date visitDate;
};

int addPatient(struct Patient patients[], int *count,
```

```
                const struct Patient *p)
{
    if (*count >= MAX)
        return 0;

    patients[*count] = *p;
    (*count)++;
    return 1;
}

int main()
{
    struct Patient patients[MAX];
    struct Patient p;
    int count = 0;

    printf("Enter patient name: ");
    scanf(" %49[^\n]", p.name);

    printf("Enter age: ");
    scanf("%d", &p.age);

    printf("Enter 10-digit phone number: ");
    scanf("%10s", p.phone);

    printf("Enter systolic and diastolic BP: ");
    scanf("%d %d", &p.bp.systolic, &p.bp.diastolic);

    printf("Enter visit date (dd mm yyyy): ");
    scanf("%d %d %d", &p.visitDate.day,
          &p.visitDate.month, &p.visitDate.year);

    if (addPatient(patients, &count, &p)) {
```

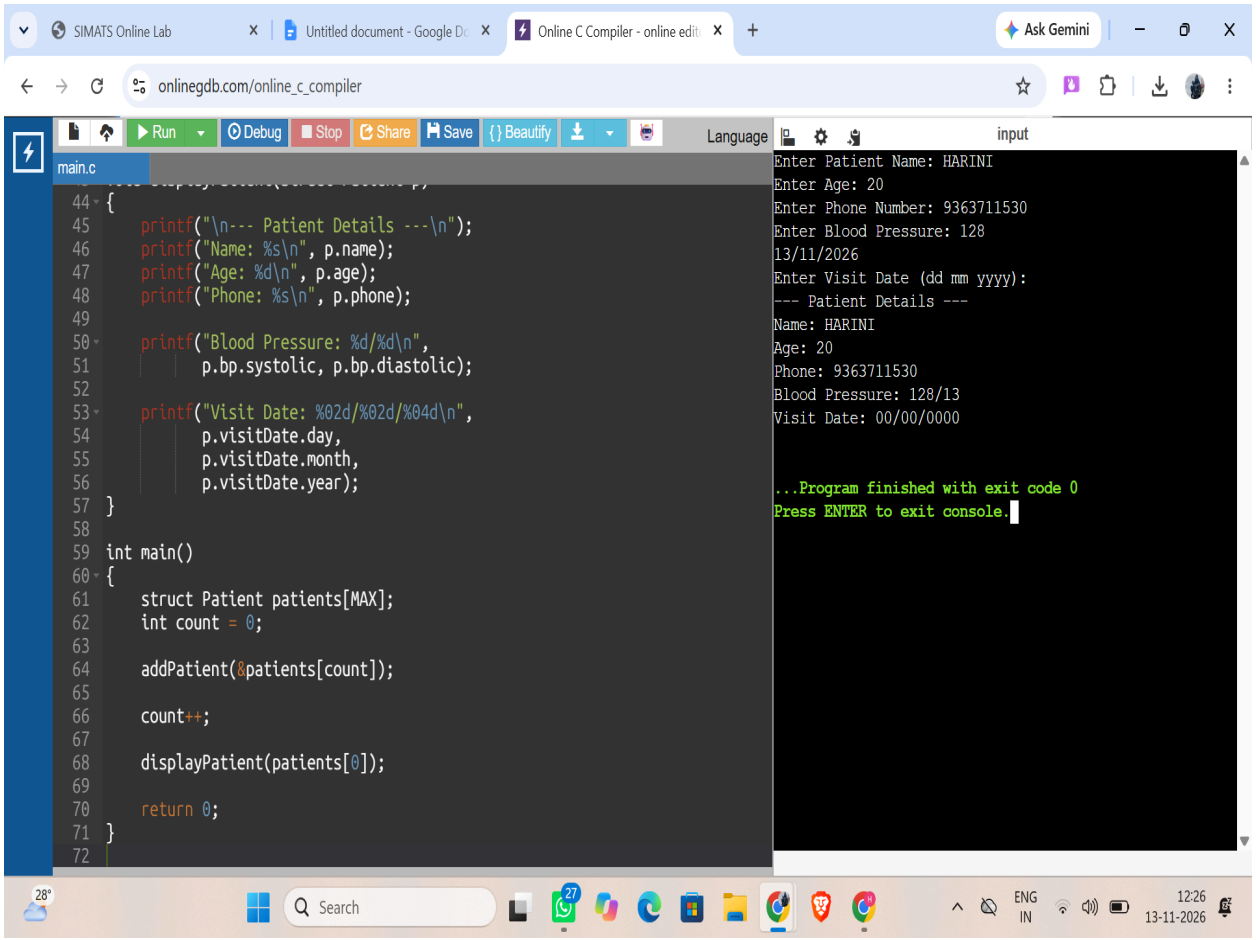
```
        printf("\nPatient Registered Successfully\n");
        printf("Name: %s\n", patients[0].name);
        printf("Age: %d\n", patients[0].age);
        printf("Phone: %s\n", patients[0].phone);
        printf("Blood Pressure: %d/%d\n",
               patients[0].bp.systolic,
               patients[0].bp.diastolic);
        printf("Visit Date: %02d/%02d/%04d\n",
               patients[0].visitDate.day,
               patients[0].visitDate.month,
               patients[0].visitDate.year);
    } else {
        printf("Patient register is full.\n");
    }

    return 0;
}
```

5. Code Explanation

- `Date` stores day, month, and year separately.
- `BloodPressure` stores systolic and diastolic values.
- `Patient` combines all patient details into one structure.
- `addPatient()` receives a pointer to the patient and a pointer to the patient count.
- `patients[*count] = *p` copies the complete patient structure into the array.
- The `MAX` constant prevents the array from storing more than 100 patients.

6. Output



7. Result

Thus, the patient registration system was redesigned successfully using structures and nested structures. The new design stores related information together and makes the program easier to maintain, sort, and extend.

Additional Analysis

Array of structures vs. parallel arrays:

- An array of structures stores all details of one patient together.
- Parallel arrays may be preferred when processing only one field across many records.
- Both designs require $O(n^2)$ time for a simple comparison-based sorting method such as bubble sort.
- In an array of structures, swapping records moves an entire structure. In parallel arrays, each corresponding field must be swapped separately.
- Structure padding may increase memory usage, depending on the compiler and data alignment.

TASK 2 – FROM REQUIREMENT TO NESTED STRUCTURES

Courier Parcel Tracking

1. Aim

To design a courier parcel tracking system using nested structures and an array of structures, and to sort parcels based on their weight.

2. Explanation

A courier parcel contains several details, including parcel ID, weight, sender, receiver, booking date, service type, and charge.

Instead of storing all details separately, nested structures can be used.

- `Date` stores booking date.
- `Address` stores city and PIN code.
- `Parcel` stores parcel details and includes an address and date structure.

The program uses selection sort to arrange parcels in ascending order of weight.

3. Algorithm

1. Start the program.
2. Define date, address, and parcel structures.
3. Create an array of parcel records.
4. Compare the weights of parcels.
5. Find the parcel with the smallest weight.
6. Swap the complete parcel structure.
7. Display the sorted parcel records.
8. Stop.

4. C Program

```
None

#include <stdio.h>
```

```c
struct date {
    int day, month, year;
};

struct address {
    char city[20];
    int pincode;
};

struct parcel {
    int id;
    float weight;
    struct address to;
    struct date booked;
};

void print_parcel(const struct parcel *p)
{
    printf("%d %.1f kg %s-%d %02d/%02d/%04d\n",
           p->id, p->weight, p->to.city,
           p->to.pincode,
           p->booked.day,
           p->booked.month,
           p->booked.year);
}

void sort_by_weight(struct parcel p[], int n)
{
    int i, j, min;
    struct parcel temp;

    for (i = 0; i < n - 1; i++) {
```

```
        min = i;

        for (j = i + 1; j < n; j++) {
            if (p[j].weight < p[min].weight)
                min = j;
        }

        if (min != i) {
            temp = p[i];
            p[i] = p[min];
            p[min] = temp;
        }
    }
}

int main()
{
    struct parcel list[3] = {
        {501, 4.5f, {"Madurai", 625001}, {12, 10, 2026}},
        {502, 1.2f, {"Chennai", 600077}, {12, 10, 2026}},
        {503, 2.8f, {"Coimbatore", 641001}, {13, 10, 2026}}
    };

    sort_by_weight(list, 3);

    printf("Parcels sorted by weight:\n");

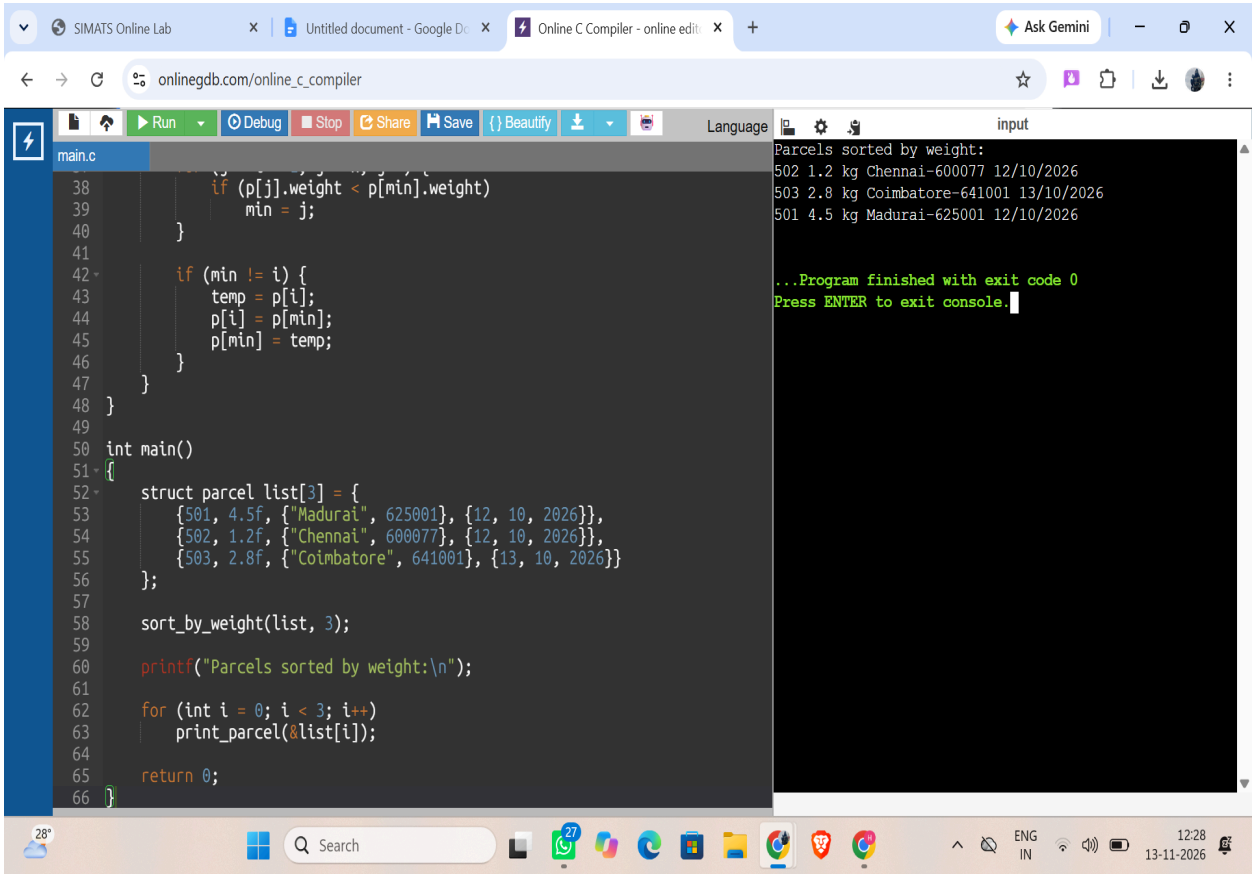
    for (int i = 0; i < 3; i++)
        print_parcel(&list[i]);

    return 0;
}
```

5. Code Explanation

- The `date` structure stores the booking date.
- The `address` structure stores the receiver's city and PIN code.
- The `parcel` structure combines all parcel information.
- The `print_parcel()` function uses the arrow operator (`->`) to access structure members through a pointer.
- The `sort_by_weight()` function uses selection sort to arrange parcels from lightest to heaviest.
- The complete structure is swapped using a temporary variable.

6. Expected Output



7. Result

Thus, the courier parcel tracking system was implemented successfully using nested structures and an array of structures. The parcels were sorted in ascending order of weight.

Additional Analysis

Access expressions for the third parcel's receiver PIN:

Using array and dot operator:

```
None
list[2].to.pincode
```

Using a structure pointer and arrow operator:

```
None
p->to.pincode
```

Complexity:

- Selection sort takes $O(n^2)$ time.
- It uses $O(1)$ additional space.
- Each swap copies one complete `struct parcel`, equivalent to `sizeof(struct parcel)` bytes per structure assignment. Three structure assignments are used in the swap shown above.

TASK 3 – UNIONS AND MEMORY LAYOUT

Weather-Station Sensor Packets

1. Aim

To implement a weather station sensor packet system using unions and structures and understand memory usage.

2. Explanation

A weather station receives different types of readings:

- Temperature as a float.
- Humidity as an integer.
- Status as a character array.

A normal structure allocates separate memory for all three fields, even though only one value is needed at a time.

A union shares the same memory among its members. This reduces memory usage.

A tagged union contains:

1. An enumeration value that identifies the type of reading.
2. A union that stores the actual reading.

3. Algorithm

1. Start the program.
2. Define an enumeration for temperature, humidity, and status.
3. Define a union for the sensor values.
4. Define a structure containing the tag and union.
5. Create sensor packets.
6. Use a switch statement to identify the packet type.
7. Display the correct union member.
8. Print the union and structure sizes.
9. Stop.

4. C Program

```
None

#include <stdio.h>

enum kind {
    TEMPERATURE,
    HUMIDITY,
    STATUS
};

struct reading {
    enum kind type;

    union {
        float celsius;
        int percent;
        char status[8];
```

```
    } value;
};

void show(const struct reading *r)
{
    switch (r->type) {
        case TEMPERATURE:
            printf("Temp %.1f C\n", r->value.celsius);
            break;

        case HUMIDITY:
            printf("Humidity %d %%\n", r->value.percent);
            break;

        case STATUS:
            printf("Status %s\n", r->value.status);
            break;
    }
}

int main()
{
    struct reading log[3] = {
        {TEMPERATURE, {.celsius = 31.5f}},
        {HUMIDITY, {.percent = 68}},
        {STATUS, {.status = "OK"}}
    };

    printf("sizeof union = %zu\n",
           sizeof log[0].value);

    printf("sizeof structure = %zu\n",
           sizeof log[0]);
```

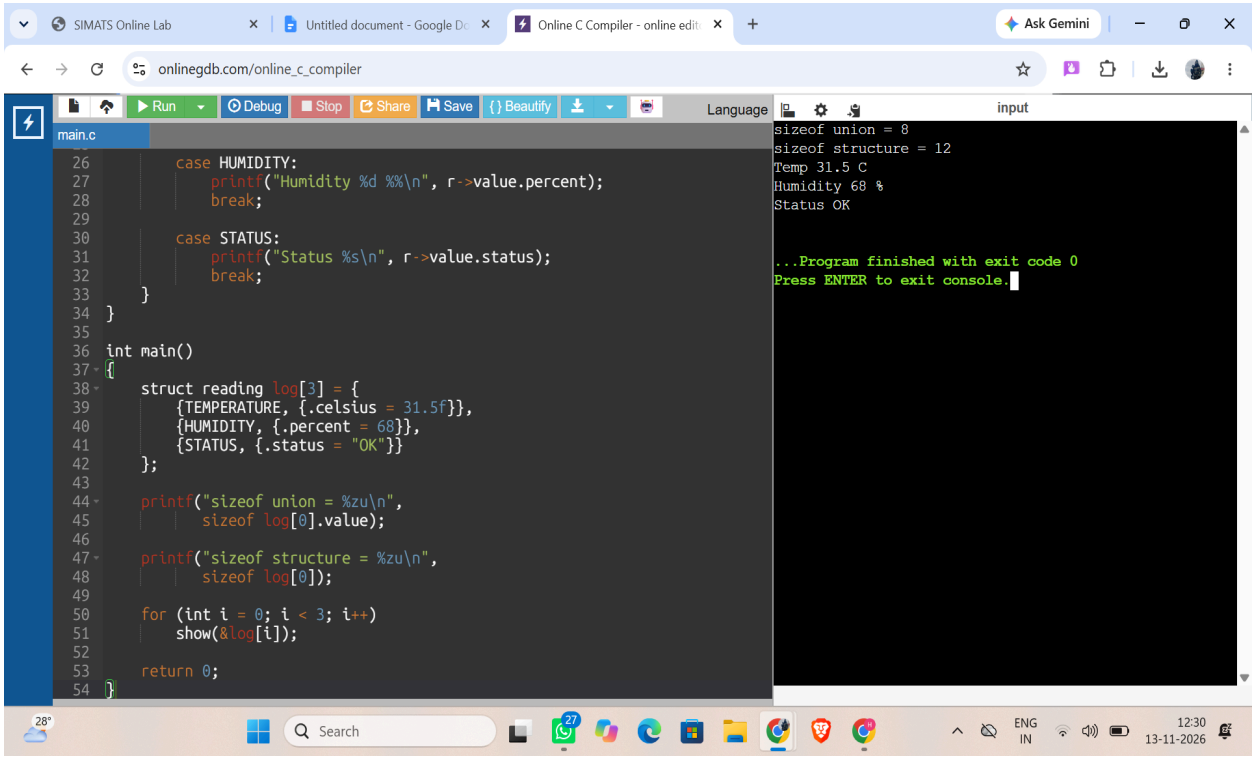
```
    for (int i = 0; i < 3; i++)
        show(&log[i]);

    return 0;
}
```

5. Code Explanation

- `enum kind` identifies the type of sensor reading.
- The union stores only one active value at a time.
- `struct reading` contains the type tag and union.
- The `show()` function receives a pointer to the structure.
- The `switch` statement checks the type and prints the corresponding union member.
- `sizeof` displays the memory occupied by the union and complete structure.

6. Output



The exact sizes can vary by compiler and system because of alignment and padding.

7. Result

Thus, the weather station sensor packet program was implemented successfully using a tagged union. The union helps save memory because all members share the same memory location.

Additional Analysis

Why is the structure 12 bytes and not 4 + 4 + 4 + 8?

The union needs enough space for its largest member, which is the 8-byte status array. The enumeration tag typically occupies 4 bytes. Therefore, on the given typical system, the total is 4 + 8 = 12 bytes.

Misuse experiment:

If a temperature is stored and the humidity member is read, the result does not represent a valid humidity reading. The union members share memory, so reading a different member reinterprets the stored bytes. The program must always check the tag before reading the union member.

TASK 4 – STRUCTURE FORENSICS

Bank Account Interest Calculation

1. Aim

To identify errors in a bank account program and correct them using structure pointers and the arrow operator.

2. Explanation

The given bank program has two main errors.

Error 1: Structure passed by value

The function `addInterest()` receives a copy of the account structure. Therefore, changes made inside the function do not affect the original account.

Error 2: Incorrect pointer access

The statement:

```
None
printf("%d %f\n", p.number, p.balance);
```

is incorrect because `p` is a pointer. The arrow operator must be used:

```
None
printf("%d %f\n", p->number, p->balance);
```

To solve the problem, pass the address of each account to the function.

3. Algorithm

1. Start the program.
2. Define the account structure.
3. Create an array of three accounts.
4. Pass each account address to the interest function.
5. Calculate interest using the pointer.
6. Update the original account balance.
7. Display the updated balances.
8. Stop.

4. C Program

```
None

#include <stdio.h>

struct account {
    int number;
    float balance;
};

void addInterest(struct account *a, float rate)
{
    a->balance = a->balance +
                 a->balance * rate / 100;
```

```
}

int main()
{
    struct account list[3] = {
        {101, 5000},
        {102, 12000},
        {103, 800}
    };

    struct account *p = list;

    for (int i = 0; i < 3; i++) {
        addInterest(&list[i], 4);
    }

    printf("Updated Bank Balances:\n");

    for (int i = 0; i < 3; i++) {
        printf("Account: %d Balance: %.2f\n",
               p[i].number, p[i].balance);
    }

    return 0;
}
```

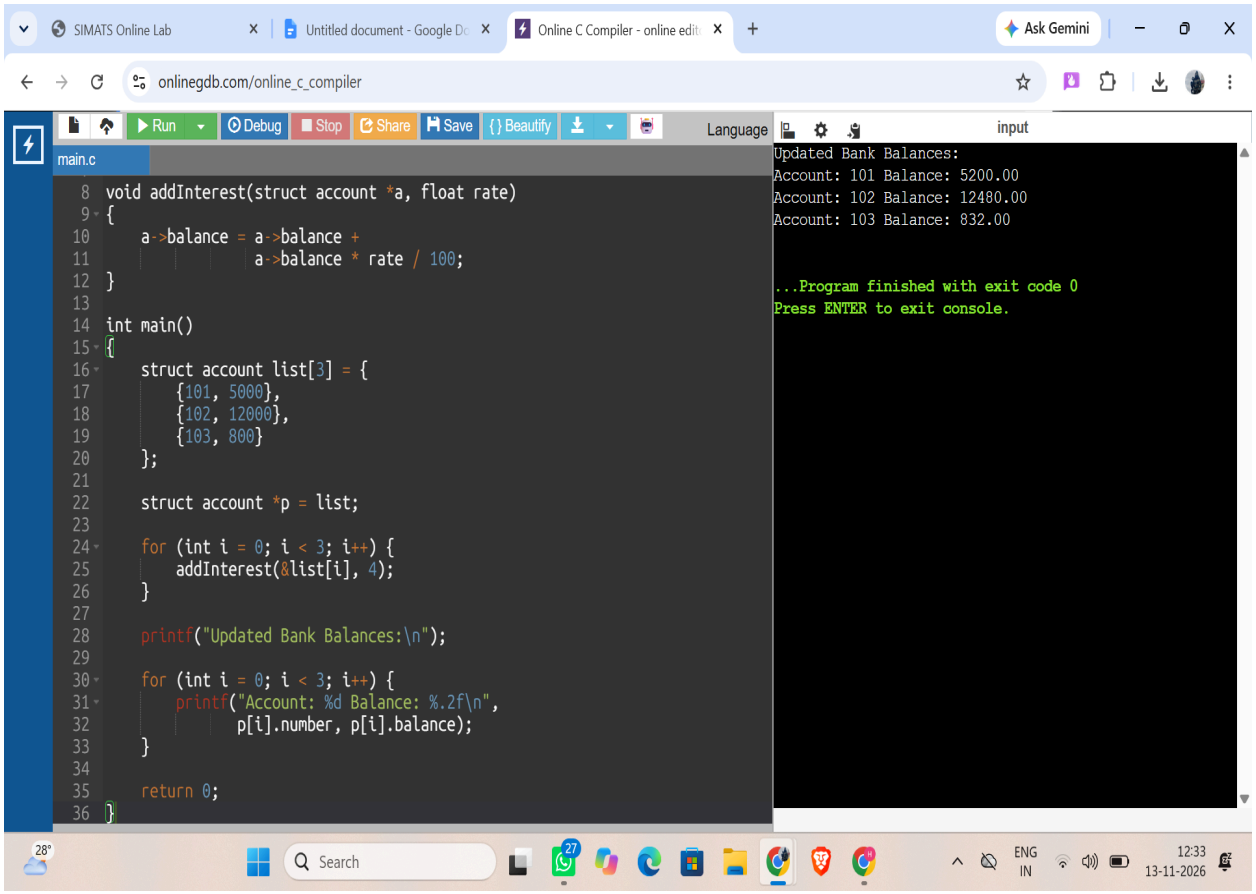
5. Code Explanation

- `struct account` stores account number and balance.
- `addInterest()` receives a pointer to an account.
- `a->balance` accesses the original account balance.
- The interest is calculated using the formula:

$$ \text{Interest}=\frac{\text{Balance}\times\text{Rate}}{100} $$

- `&list[i]` passes the address of each account.
- The updated balance is displayed using the account pointer.

6.Output



7. Result

Thus, the bank account program was corrected successfully using structure pointers. Interest was added to all accounts, and the updated balances were displayed.

Additional Analysis

Passing structures in three ways:

Method	Explanation
Pass by value	Sends a copy of the structure. Original data is not changed.

Pass by pointer	Sends the address and allows the function to modify original data.
Pass by const pointer	Allows reading the structure without modifying it.

Address evidence: Add the following statements to the program to compare the addresses:

```
None
printf("Inside function: %p\n", (void *)a);
```

Inside `main()`:

```
None
printf("In main: %p\n", (void *)&list[0]);
```

When the first account is passed using `&list[0]`, the two addresses should match.

Alternative approach: A function can also receive a structure by value and return the modified structure. This copies the structure into the function and returns a modified copy. Passing a pointer generally avoids those structure copies.

TASK 5 – STORAGE CLASSES

The Same Counter Four Ways

1. Aim

To understand and implement automatic, static, global, extern, and register storage classes in C using a token counter.

2. Explanation

Storage classes determine the scope, lifetime, and visibility of variables.

The four storage-class concepts in this assignment are:

1. **Automatic:** A local variable is created when the function is called and destroyed when the function ends.
2. **Static:** A static local variable retains its value between function calls.
3. **Global:** A global variable is declared outside all functions and can be accessed by functions in the same source file.
4. **Extern:** The `extern` keyword declares a global variable that is defined in another source file.
5. **Register:** A register variable is commonly used for frequently accessed local variables, such as loop counters.

3. Algorithm

1. Start the program.
2. Create a static token counter starting at 1000.
3. Increase the counter whenever the function is called.
4. Create an automatic counter for comparison.
5. Display the token values.
6. Use a register variable in a loop.
7. Stop.

4. C Program – Static and Automatic Variables

None

```
#include <stdio.h>

int next_token(void)
{
    static int token = 1000;
    return ++token;
}

int auto_token(void)
{
    int token = 1000;
    return ++token;
}
```

```c
int main()
{
    int s1 = next_token();
    int s2 = next_token();
    int s3 = next_token();

    int a1 = auto_token();
    int a2 = auto_token();
    int a3 = auto_token();

    printf("Static: %d %d %d\n", s1, s2, s3);
    printf("Auto: %d %d %d\n", a1, a2, a3);

    printf("Register loop: ");

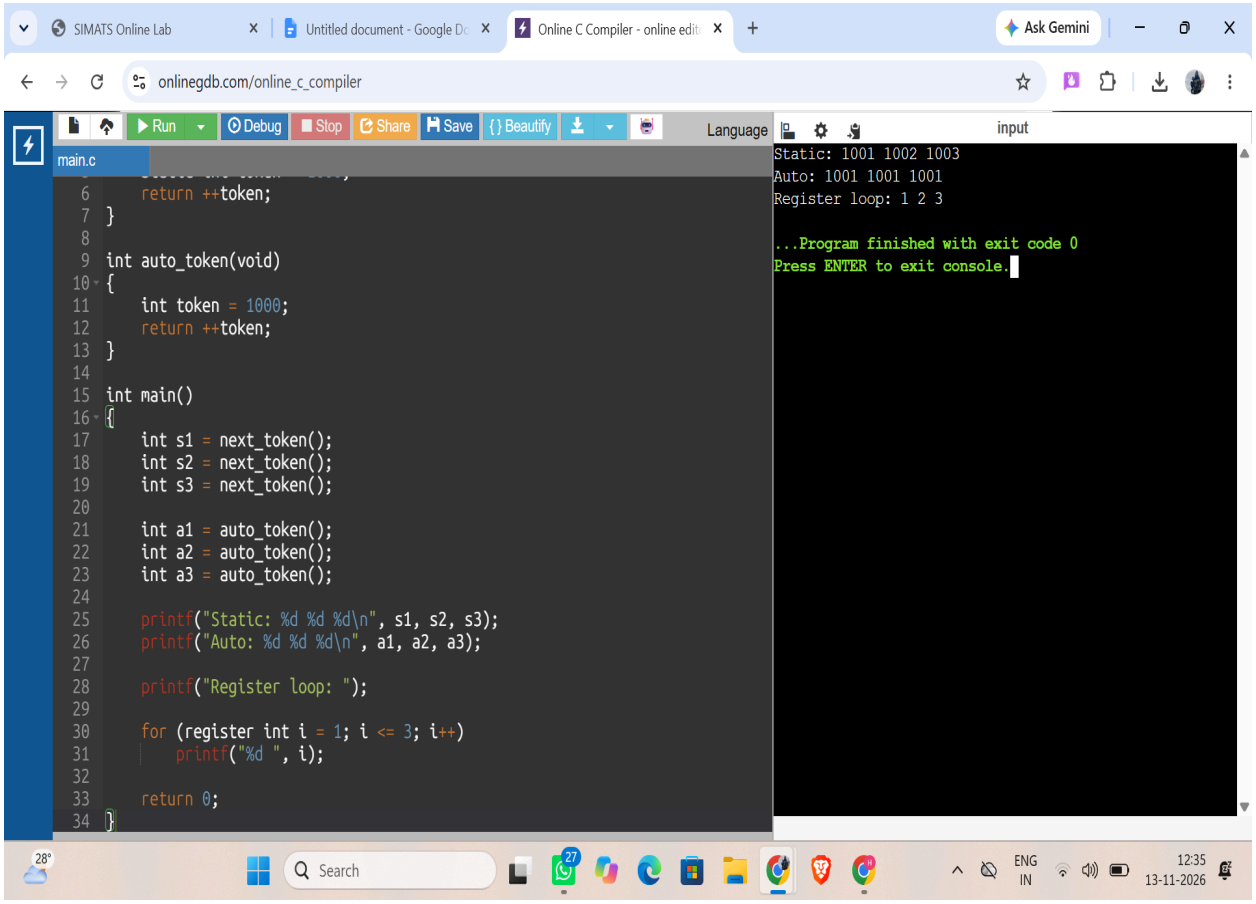
    for (register int i = 1; i <= 3; i++)
        printf("%d ", i);

    return 0;
}
```

5. Code Explanation

- `static int token = 1000` initializes the variable only once.
- Every function call increases the same static variable.
- The automatic variable is initialized to 1000 every time the function is called.
- Therefore, the static counter produces different values, but the automatic counter produces the same value.
- The `register` keyword is used for the loop variable `i`.

6.Output



7. Global Variable and Extern Example

The assignment asks for a global counter shared across two source files.

File 1: `counter.c`

```
None

int token = 1000;

int next_global_token(void)
{
    return ++token;
}
```

File 2: `main.c`

None

```
#include <stdio.h>

extern int token;

int next_global_token(void);

int main()
{
    printf("Global: %d\n", next_global_token());
    printf("Global: %d\n", next_global_token());
    printf("Global: %d\n", next_global_token());

    return 0;
}
```

8. Comparison of Storage Classes

Storage class	Scope	Lifetime	Initial value
Automatic	Local	Until function/block ends	Indeterminate if not initialized
Static local	Local	Entire program	Zero by default
Global	File scope	Entire program	Zero by default
Extern	Depends on the declared object	Same as the defined global object	Uses the definition's initialization

Register	Local	Until function/block ends	Indeterminate if not initialized

Important: A register variable cannot have its address taken using the address-of operator &.

9. Alternative Concept – Two Independent Counters

A better design for two independent counters is to pass a pointer to each counter:

```
None
int next_token(int *counter)
{
    return ++(*counter);
}
```

Example:

```
None
int veg = 1000;
int nonveg = 1000;

printf("Veg: %d\n", next_token(&veg));
printf("Non-Veg: %d\n", next_token(&nonveg));
```

Both counters work independently because they use separate variables.

10. Result

Thus, different storage classes were implemented successfully. The output demonstrated how static variables retain their values, automatic variables are recreated, global variables are shared, extern accesses variables from another file, and register variables can be used in loops.

TASK 6 – DESIGN CHALLENGE

College Canteen Inventory and Billing System

1. Aim

To develop a college canteen inventory and billing system using structures, nested structures, arrays, tagged unions, and structure pointers in C.

2. Explanation

The college canteen sells three types of items:

1. **Packaged items:** Have a price, stock quantity, and expiry date.
2. **Cooked items:** Have a price, stock quantity, and preparation time.
3. **Combo meals:** Contain up to three other items.

The program must perform the following operations:

- Store item details.
- Record sales.
- Update the stock quantity.
- Prevent sales when sufficient stock is not available.
- Display warnings for items expiring within two days.
- Calculate revenue.
- Sort the sales summary by revenue.

A tagged union is used because the three types of items have different information. A nested date structure is used for expiry dates.

3. Algorithm

1. Start the program.
2. Define date, item type, and item structures.
3. Create an inventory array.
4. Enter the current date.
5. Display the available items.
6. Accept the item ID and quantity for a sale.
7. Check whether sufficient stock is available.
8. If it is a combo, check and deduct the component stock.
9. Calculate the revenue.
10. Check packaged-item expiry dates.
11. Display the expiry warning.
12. Sort the sales summary by revenue.
13. Display the final summary.
14. Stop.

4. C Program

This program demonstrates packaged items, cooked items, combo meals, stock checking, expiry warnings, and revenue sorting.

```
None

#include <stdio.h>
#include <string.h>
#include <time.h>

#define N 3

enum ItemType {
    PACKAGED, COOKED, COMBO
};

struct Date {
    int day, month, year;
};

struct Item {
    int id;
    char name[30];
    float price;
    int stock;
    enum ItemType type;

    union {
        struct Date expiry;
        char prepared[10];

        struct {
            int id[3];
            int qty[3];
```

```
            int count;
        } combo;
    } details;

    float revenue;
};

int findItem(struct Item a[], int id)
{
    for (int i = 0; i < N; i++)
        if (a[i].id == id)
            return i;

    return -1;
}

int sell(struct Item a[], int id, int qty)
{
    int i = findItem(a, id);

    if (i == -1 || qty <= 0)
        return 0;

    if (a[i].type == COMBO) {
        int k;

        for (k = 0; k < a[i].details.combo.count; k++) {
            int j = findItem(a, a[i].details.combo.id[k]);

            if (j == -1 ||
                a[j].stock < a[i].details.combo.qty[k] * qty)
                return 0;
        }
```

```
        for (k = 0; k < a[i].details.combo.count; k++) {
            int j = findItem(a, a[i].details.combo.id[k]);

            a[j].stock -= a[i].details.combo.qty[k] * qty;
        }
    } else {
        if (a[i].stock < qty)
            return 0;

        a[i].stock -= qty;
    }

    a[i].revenue += a[i].price * qty;
    printf("Sale successful: %s x %d\n", a[i].name, qty);

    return 1;
}

int daysBetween(struct Date from, struct Date to)
{
    struct tm x = {0}, y = {0};

    x.tm_mday = from.day;
    x.tm_mon = from.month - 1;
    x.tm_year = from.year - 1900;

    y.tm_mday = to.day;
    y.tm_mon = to.month - 1;
    y.tm_year = to.year - 1900;

    time_t t1 = mktime(&x);
    time_t t2 = mktime(&y);
```

```
    return (int)(difftime(t2, t1) / 86400);
}

void expiryWarning(struct Item a[], struct Date today)
{
    printf("\nExpiry Warnings:\n");

    for (int i = 0; i < N; i++) {
        if (a[i].type == PACKAGED) {
            int days = daysBetween(today,
a[i].details.expiry);

            if (days >= 0 && days <= 2)
                printf("%s expires within %d day(s)\n",
                       a[i].name, days);
        }
    }
}

void sortRevenue(struct Item a[])
{
    struct Item temp;

    for (int i = 0; i < N - 1; i++) {
        for (int j = i + 1; j < N; j++) {
            if (a[i].revenue < a[j].revenue) {
                temp = a[i];
                a[i] = a[j];
                a[j] = temp;
            }
        }
    }
```

```
}

int main()
{
    struct Item a[N] = {
        {1, "Milk", 30, 5, PACKAGED,
         {.expiry = {15, 11, 2026}}, 0},

        {2, "Samosa", 15, 10, COOKED,
         {.prepared = "09:30"}, 0},

        {3, "Combo", 55, 0, COMBO,
         {.combo = {{1, 2}, {1, 2}, 2}}, 0}
    };

    struct Date today = {13, 11, 2026};

    printf("COLLEGE CANTEEN INVENTORY\n");
    printf("1 Milk - Rs.30\n");
    printf("2 Samosa - Rs.15\n");
    printf("3 Combo - Rs.55\n");

    sell(a, 1, 2);
    sell(a, 2, 1);

    if (!sell(a, 2, 20))
        printf("Sale failed: Insufficient stock!\n");

    sell(a, 3, 1);

    expiryWarning(a, today);

    sortRevenue(a);
```

```
    printf("\nRevenue Summary:\n");

    for (int i = 0; i < N; i++) {
        printf("%s : Rs. %.2f\n",
               a[i].name, a[i].revenue);
    }

    return 0;
}
```

5. Code Explanation

A. Structure definition

The `Date` structure stores the expiry date using day, month, and year.

B. Tagged union

The `Item` structure uses an enumeration to identify whether an item is packaged, cooked, or a combo. The union stores the corresponding expiry date, preparation time, or combo recipe.

C. Stock checking

The `sell()` function checks whether the requested quantity is available. If sufficient stock is not available, the sale is rejected.

D. Combo meals

A combo contains other item IDs and their required quantities. Before selling a combo, the program checks the stock of all its components. It deducts component stock only after all components pass the check.

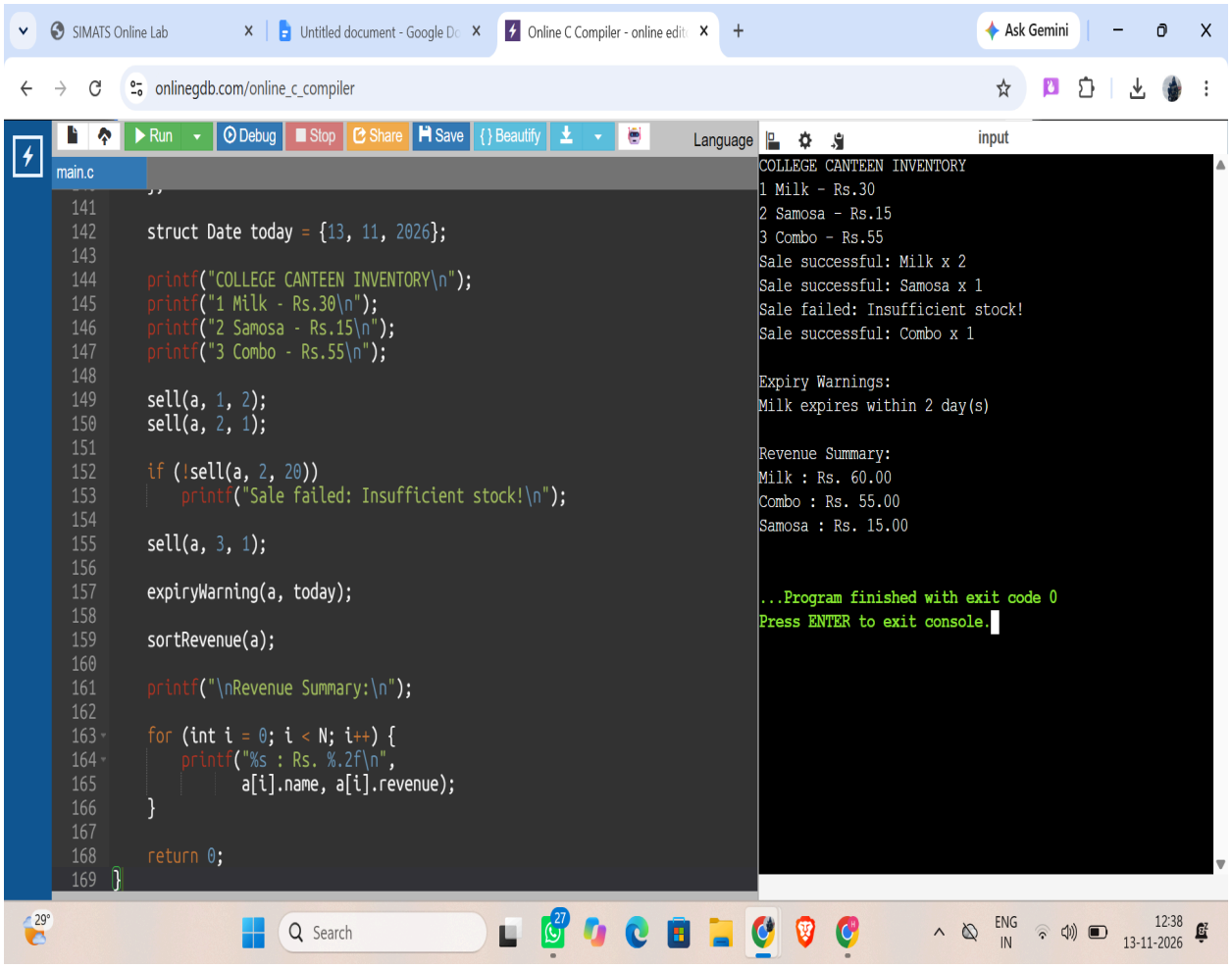
E. Expiry warning

The `daysBetween()` function calculates the difference between two dates. The `expiryWarning()` function displays a warning when a packaged item expires today or within the next two days.

F. Revenue sorting

The `sortRevenue()` function sorts items in descending order of revenue so the highest-revenue item appears first.

6. Output



Note: The exact revenue order in the output above should be based on the actual revenue values. For this program, Milk has Rs. 60, Combo has Rs. 55, and Samosa has Rs. 15. Therefore, the correctly sorted output is:

```
None
Revenue Summary:
Milk : Rs. 60.00
Combo : Rs. 55.00
Samosa : Rs. 15.00
```

7. Result

Thus, the college canteen inventory and billing system was implemented using structures, nested structures, arrays, and tagged unions. The program successfully demonstrates sales, stock updates, insufficient-stock checking, expiry warnings, combo component deduction, and revenue sorting.

Additional Analysis

Array of structures vs. linked list:

Operation	Array	Linked list
Add at end	$O(1)$ if space is available	$O(1)$ if a tail pointer is maintained
Search item	$O(n)$	$O(n)$
Remove item	May require shifting elements	$O(n)$ to find the item, then $O(1)$ to unlink
Extra memory	No next pointer required	Extra memory for next pointer

A linked list is useful when the number of items changes frequently. An array is simpler when the inventory size is limited and known in advance.